

Accord — Threat Model and Safety Model

Status

Draft v0.1

Purpose

This document defines the threat model and safety posture for Accord v1.

Accord is not a general security platform, but it is explicitly a **trust-sensitive control layer**. That means the first version must be clear about:

- what can go wrong,
- what failures matter most,
- how the system is expected to behave when something is missing or invalid,
- what is in scope for v1,
- and what risks are deliberately left for later versions.

This document is not a formal security certification artifact. It is a practical engineering threat model for building a local-first approval-aware control plane.

1. Security and Safety Philosophy

Accord's core promise is not that it makes AI harmless.

Its promise is narrower and more practical:

If an AI application uses personal context and wants to take a meaningful action, Accord should make that behavior more explicit, more inspectable, and harder to execute silently when policy or approval requirements are not satisfied.

That means v1 should prioritize:

- explicit boundaries,
- fail-closed execution,
- visible runtime decisions,
- append-only history,
- and low ambiguity around approval state.

It should not overclaim.

2. Protection Goals

Accord v1 should protect the following properties as strongly as possible within a local-first architecture.

2.1 Context Use Transparency

A user or developer should be able to see what categories of context influenced an action proposal.

2.2 Approval Integrity

If an action requires approval, it should not execute without a valid approval result.

2.3 Scope Integrity

A granted scope should only be usable within its allowed categories, time window, and status.

2.4 Revocation Effectiveness

Once a scope or approval is revoked, future use must be blocked.

2.5 Auditability

Important state transitions should leave a durable, inspectable record.

2.6 Predictable Failure

When information is missing, stale, revoked, denied, or invalid, the system should fail closed rather than guess.

3. Assets Worth Protecting

In Accord, the main assets are not just raw secrets. They are the runtime trust artifacts that control whether action should proceed.

Core Assets

- `ScopeRequest` objects
- `GrantedScope` objects
- `ContextManifest` objects
- `ActionProposal` objects
- `ApprovalReceipt` objects
- `RevocationEvent` objects
- `AuditRecord` history
- local policy rules and configs
- local user context references exposed through Accord flows

Why these matter

If these objects are corrupted, forged, bypassed, or misrepresented, the system may appear trustworthy while allowing unsafe behavior.

4. Trust Boundaries

The architecture document already defined runtime boundaries. The threat model now names what can go wrong at each one.

Boundary A — Context Access Boundary

This is where scope is requested and granted.

Main Risks

- requesting more context than necessary
- granting overly broad scope
- using expired scope
- continuing to use revoked scope
- ambiguity about what categories were actually granted

Boundary B — Context Disclosure Boundary

This is where the system declares what it used.

Main Risks

- incomplete disclosure of used context
- misleading summaries
- omission of sensitive influencing context
- mismatch between actual context use and manifest contents

Boundary C — Action Approval Boundary

This is where risky action should pause.

Main Risks

- action proceeds without approval
- approval is obtained for one target but reused for another
- approval expires but execution still occurs
- a denied proposal is retried or replayed without clear treatment

Boundary D — Execution Boundary

This is where side effects happen.

Main Risks

- host application bypasses verification
- executor trusts stale or forged receipt data
- action payload differs from what was approved
- execution target changes after approval

Boundary E — Audit Boundary

This is where history becomes durable.

Main Risks

- audit records missing for meaningful transitions
 - audit history overwritten or silently mutated
 - unclear linkage between proposal, receipt, and execution
-

5. Threat Categories

For Accord v1, I would group threats into seven practical categories.

5.1 Scope Abuse

The application requests or uses more context than needed.

Examples: - requesting unrelated categories “just in case” - retaining valid scope longer than intended - using one granted scope for a different purpose than stated

5.2 Disclosure Mismatch

The context manifest does not accurately describe what influenced the action.

Examples: - missing entries in the manifest - overly vague summaries - hidden context inputs that affected the proposal

5.3 Approval Bypass

A side effect is executed even though approval was required and not valid.

Examples: - executor never checks receipt validity - stale approval is accepted - denied proposal still reaches execution path

5.4 Replay / Reuse Misuse

A valid decision object is reused outside the intended action boundary.

Examples: - single approval reused multiple times - approval for one target reused for another target - old scope reused after workflow context has changed

5.5 Audit Integrity Failure

The system cannot clearly reconstruct what happened.

Examples: - proposal exists but receipt is missing - receipt exists but no corresponding proposal is stored - audit records are mutable without trace

5.6 Host Application Misbehavior

The integrating app behaves incorrectly or dishonestly.

Examples: - app generates incomplete manifests - app executes actions outside Accord flow - app tampers with proposal semantics after approval

5.7 Policy Misconfiguration

The local policy rules are too permissive, contradictory, or unclear.

Examples: - risky actions accidentally set to auto-allow - stale configs leave approval disabled - category matching is broader than intended

6. Primary Threat Scenarios

This section defines the most important concrete threat scenarios for v1.

Scenario 1 — Execution Without Required Approval

Description

An action classified as `require_approval` still executes due to bad integration or verification gaps.

Why it matters

This is a direct violation of Accord's core product promise.

Expected v1 Mitigations

- executor must verify a valid `ApprovalReceipt`

- verification must fail if decision is not `approved`
- verification must fail if receipt is expired, revoked, or target-mismatched
- sample integrations must demonstrate fail-closed behavior
- audit record should show proposal, decision, and execution attempt outcome

Residual Risk

A malicious host application can still bypass Accord entirely by refusing to use it. Accord cannot solve dishonesty outside its control path in v1.

Scenario 2 — Approval for One Target Reused for Another

Description

An approval granted for one resource or action target is replayed against a different target.

Why it matters

This breaks the meaning of user approval.

Expected v1 Mitigations

- support `target_lock` or equivalent in `ApprovalReceipt`
- executor verification must compare proposal target to receipt conditions
- sample apps should use single-use approval by default where practical

Residual Risk

Some host integrations may define targets too loosely. This is partly a protocol ergonomics issue and partly an adapter responsibility.

Scenario 3 — Expired or Revoked Scope Still Used

Description

An app continues using a scope after expiry or revocation.

Why it matters

This undermines user control over context access.

Expected v1 Mitigations

- all scope use must re-check status
- policy engine must deny expired or revoked scope

- revocation must be first-class and durable
- console should surface active vs expired vs revoked clearly

Residual Risk

Applications caching context outside Accord's awareness remain a limitation in v1.

Scenario 4 — Misleading or Incomplete Context Manifest

Description

The action proposal's manifest omits relevant context or describes it in a misleading way.

Why it matters

The user may approve an action without understanding what influenced it.

Expected v1 Mitigations

- protocol requires manifest attachment for meaningful actions
- `used_entries` should be human-legible
- sample apps should demonstrate specific and concrete summaries
- audit trail should preserve manifest alongside proposal

Residual Risk

Accord v1 cannot cryptographically prove that a host app disclosed every influencing input. This is a known limitation.

Scenario 5 — Audit Trail Gaps

Description

Meaningful transitions happen but no durable record exists.

Why it matters

Without a reliable trail, the system cannot support inspection or debugging.

Expected v1 Mitigations

- append audit record on every major object creation and state transition
- store audit records separately from transient UI state
- console should make linkage between objects easy to inspect

Residual Risk

SQLite or local storage tampering is not fully prevented in v1. We can make mutation visible by convention, but not impossible.

Scenario 6 — Overly Broad Policy Configuration

Description

A local policy file allows too much by default.

Why it matters

The system may appear to provide safety while silently allowing risky flows.

Expected v1 Mitigations

- conservative default policies
- explicit treatment of high/critical risk actions
- documented sample policies that require approval for meaningful writes
- configuration validation where possible

Residual Risk

A user can intentionally loosen policy. That is acceptable if the behavior is explicit.

7. Attacker / Failure Sources

Accord v1 should assume threats can come from multiple sources.

7.1 Honest-but-buggy host application

The integrating app intends to use Accord correctly but has logic bugs.

7.2 Over-permissive developer integration

The developer integrates quickly but makes unsafe choices in configuration or target modeling.

7.3 Curious or careless local user

A user changes policy or state without fully understanding the effects.

7.4 Malicious local process or app

A local process tampers with files, state, or execution flow.

7.5 Dishonest integrating application

An application pretends to use Accord semantics but bypasses or misrepresents behavior.

The first four are realistic v1 concerns. The fifth is important because Accord is a control layer, not a trusted execution environment.

8. Assumptions and Limits of v1

Being explicit about assumptions matters.

Assumption A

The host application chooses to integrate through Accord rather than bypassing it.

Assumption B

The local environment is not a hardened hostile environment.

Assumption C

The initial storage model is local and inspectable, not tamper-proof against a determined local attacker.

Assumption D

The first version optimizes for clarity and trustworthy local flows, not enterprise-grade cryptographic guarantees.

These assumptions do not weaken the project. They keep the claims honest.

9. Safety Requirements for v1

These are practical must-haves.

SR1 — Fail Closed on Missing Approval

If approval is required and no valid receipt exists, execution must not proceed.

SR2 — Fail Closed on Expired or Revoked State

Expired or revoked scope or receipts must not authorize future action.

SR3 — Require Explicit Proposal Before Meaningful Execution

Meaningful side effects must have an `ActionProposal` in the Accord flow before execution.

SR4 — Preserve Linked History

Proposal, manifest, scope, receipt, and audit history should remain linked and queryable.

SR5 — Default to Conservative Policy

Meaningful writes should require approval by default in sample policies.

SR6 — No Silent Auto-Execution for Approval-Required Actions

The reference implementation must not blur policy-required approvals into silent execution shortcuts.

SR7 — Make Status Legible

Pending, approved, denied, expired, and revoked states must be clearly represented.

10. Safety Model for Runtime Decisions

Accord should follow a simple decision doctrine:

If state is valid and policy says allow

Proceed without human approval.

If state is valid and policy says require approval

Pause until a valid approval receipt exists.

If state is invalid, missing, stale, expired, revoked, or denied

Do not proceed.

This sounds obvious, but spelling it out helps avoid hidden ambiguity in implementation.

11. Secure Defaults for v1

The reference implementation should ship with strong defaults.

Recommended Defaults

- local-only server binding by default
- approval required for file writes, commits, pushes, publishing, and external side effects
- receipts single-use by default where practical
- scopes time-bounded by default
- append-only audit history by convention
- explicit expiry surfaced in console
- no broad “allow all” starter policy

These defaults matter because many early adopters will copy sample configurations.

12. What Accord v1 Does Not Protect Against

This section is important so the project does not overclaim.

Accord v1 does **not** fully protect against:

- a malicious host application that bypasses Accord entirely
- a compromised local machine
- a determined attacker who directly tampers with local storage
- perfect proof that the context manifest includes every influencing signal
- identity spoofing across distributed environments
- cross-device synchronization integrity
- enterprise insider threat scenarios
- cryptographic non-repudiation of approvals

These are future concerns, not v1 guarantees.

13. Future Safety Enhancements Beyond v1

These are valuable, but deliberately deferred.

Potential Future Enhancements

- cryptographic signing of receipts
- tamper-evident audit chains
- stronger receipt binding to payload hashes
- distributed approval delegation
- field-level context redaction

- richer provenance guarantees
- policy simulation and linting
- signed adapters / trusted executor model
- remote secure storage and encrypted sync

The first version should not try to solve all of these at once.

14. Safety Test Ideas

The threat model should influence testing from the start.

Example Test Categories

- action blocked when approval required but missing
- action blocked when receipt expired
- action blocked when scope revoked
- action blocked when target lock mismatched
- action denied when policy rules say deny
- audit records created for all state changes
- console reflects pending vs approved vs denied accurately
- replay attempt with single-use receipt fails

These can become the basis for the early regression suite.

15. Documentation Warnings for Integrators

The SDK and docs should warn integrators clearly:

- Accord does not secure actions that bypass Accord flow
- approval receipts must be verified immediately before side effects
- context manifests should be specific and human-readable
- high-risk actions should be modeled explicitly, not hidden under generic action types
- if in doubt, require approval rather than auto-allow

These warnings should appear in the README and sample integrations.

16. Threat Model Summary

If I reduce the whole threat model to the most important line, it is this:

Accord's credibility depends on making it difficult to execute meaningful actions silently when context use, approval state, or policy conditions do not support that execution.

That is the center of the safety model.

17. Closing Statement

Accord v1 should not pretend to be a complete security system.

It should be a disciplined, local-first control layer with explicit trust boundaries, conservative defaults, and clear fail-closed behavior.

If we implement those properties honestly, the project will already be significantly more trustworthy than the ad hoc approval and context flows many AI applications rely on today.